

Criar Dockerfiles

Para construir imagens Docker, primeiro você precisa de um Dockerfile. Um Dockerfile é um arquivo de texto chamado Dockerfile sem extensão.

```
FROM <baseimage> # obtenha-as em https://hub.docker.com

WORKDIR /app # define o diretório de trabalho dentro da imagem

# copia o código-fonte da máquina host para a imagem (host: sua
# máquina de desenv. ou servidor)
# o primeiro ponto (.) é o diretório de trabalho atual (CWD) do
# host, o segundo ponto é o CWD da imagem
COPY . .

# executa qualquer comando shell dentro da imagem (ex: npm run
# build)
RUN <comando>

# na verdade não expõe a porta 80 (apenas documentação) mas
# serve para sabermos qual porta expor
EXPOSE 80 # expomos portas usando a flag --publish ao rodar

# este comando é executado quando o contêiner inicia (ex:
# ["node", "server.js"])
CMD <comando>
```

Criar Dockerfiles de Múltiplos Estágios

Dockerfiles Multistage são usados para otimização. Um caso comum é usar um estágio para compilar/gerar arquivos (builder) e outro menor para rodá-los. Diminui o tamanho final e a superfície de ataque pois não leva ferramentas de compilação.

```
# usando "as", você dá um nome (apelido) ao estágio atual
FROM <baseimage> as builder

# agora instale dependências, compile seu código, etc.
RUN <comando> # ex: g++ main.cpp

# o segundo estágio pode usar uma imagem bem menor (ex: alpine)
# ou vazia (FROM scratch)
# se não precisar de nenhuma biblioteca do sistema operacional:
FROM scratch as serve

# copia arquivos do estágio de build (ex: o executável
# compilado)
COPY --from=builder ./hello-world ./hello-world

# comando executado na inicialização do contêiner
CMD ["/hello-world"]
```

Criar Imagens Docker

```
# <caminho> define o contexto para o comando docker build
# se usar o ponto (.), usará o diretório atual do host para
# achar o Dockerfile
$ docker build <caminho>

# se o Dockerfile estiver em outro lugar ou com outro nome, use
# --file
# note que o docker ainda usará o <caminho> no final como
# contexto de arquivos
$ docker build --file ./caminho/do/Dockerfile ./caminho/do/
# contexto

# nomeando imagens: <nome> equivale a <nome>:<tag>
# o nome geralmente é estruturado como <usuario>/
# <repositorio>:<versao>
$ docker build --tag usuario/repo:0.1.0 .

# listar todas as imagens disponíveis localmente
$ docker image ls
```

Criar Contêineres Docker

```
# Contêineres Docker são instâncias de imagens em execução
$ docker run <nome-da-imagem>

# você pode rodar imagens públicas do Docker Hub ou de
# registros privados
$ docker run https://registroprivado.com/<nome-da-imagem>

# contêineres iniciam em primeiro plano (foreground). Se fechar
# o terminal, eles param.
# para rodar em segundo plano, use o modo "detached" (-d)
$ docker run --detached <nome-da-imagem>

# listar todos os contêineres ativos
$ docker container ls
# ou sintaxe abreviada antiga:
$ docker ps

# listar todos os contêineres (incluindo os parados)
$ docker container ls --all

# parar um contêiner em execução
$ docker stop <container-id>

# remover um contêiner (apenas contêineres parados podem ser
# removidos)
$ docker rm <container-id>

# iniciar um contêiner que já estava parado
$ docker start <container-id>

# reiniciar um contêiner ativo
$ docker restart <container-id>

# remover automaticamente o contêiner quando ele for parado
$ docker run --rm <nome-da-imagem>
```

Acessar Contêineres Docker

```
# publica portas. Ex: encaminha a porta do host para a porta do
# contêiner
$ docker run --publish <porta-host>:<porta-container> <nome-da-
# imagem>

# executa um comando shell dentro de um contêiner em execução
$ docker exec --interactive --tty <container-id> <comando>

# abre um terminal interativo (como se estivesse conectando via
# SSH ao contêiner)
$ docker exec --interactive --tty <container-id> sh

# sair do terminal do contêiner
$ exit
```

Criar Volumes Docker

Para persistir dados entre reinicializações e remoções de contêineres, você precisa de volumes. Sem eles, as alterações de dados são perdidas ao excluir o contêiner.

```
# usando um volume nomeado (o docker gerencia o local no host
# automaticamente)
$ docker run --volume <nome-do-volume>:/caminho/no/container
# <nome-da-imagem>

# usando um volume montado / Bind Mount (você define o
# diretório do seu host)
$ docker run --volume /caminho/no/host:/caminho/no/container
# <nome-da-imagem>

# lista todos os volumes criados no sistema
$ docker volume ls
```